1 Introduction

Objectives

The objectives of this chapter are to introduce software engineering and to provide a framework for understanding the rest of the book. When you have read this chapter, you will:

- understand what software engineering is and why it is important;
- know the answers to key questions that provide an introduction to software engineering;
- understand some ethical and professional issues that are important for software engineers.

Contents

1.1 FAQs about software engineering
1.2 Professional and ethical responsibility

Virtually all countries now depend on complex computer-based systems. National infrastructures and utilities rely on computer-based systems and most electrical products include a computer and controlling software. Industrial manufacturing and distribution is completely computerised, as is the financial system. Therefore, producing and maintaining software cost-effectively is essential for the functioning of national and international economies.

Software engineering is an engineering discipline whose focus is the cost-effective development of high-quality software systems. Software is abstract and intangible. It is not constrained by materials, or governed by physical laws or by manufacturing processes. In some ways, this simplifies software engineering as there are no physical limitations on the potential of software. However, this lack of natural constraints means that software can easily become extremely complex and hence very difficult to understand.

The notion of *software engineering* was first proposed in 1968 at a conference held to discuss what was then called the 'software crisis'. This software crisis resulted directly from the introduction of new computer hardware based on integrated circuits. Their power made hitherto unrealisable computer applications a feasible proposition. The resulting software was orders of magnitude larger and more complex than previous software systems.

Early experience in building these systems showed that informal software development was not good enough. Major projects were sometimes years late. The software cost much more than predicted, was unreliable, was difficult to maintain and performed poorly. Software development was in crisis. Hardware costs were tumbling whilst software costs were rising rapidly. New techniques and methods were needed to control the complexity inherent in large software systems.

These techniques have become part of software engineering and are now widely used. However, as our ability to produce software has increased, so too has the complexity of the software systems that we need. New technologies resulting from the convergence of computers and communication systems and complex graphical user interfaces place new demands on software engineers. As many companies still do not apply software engineering techniques effectively, too many projects still produce software that is unreliable, delivered late and over budget.

I think that we have made tremendous progress since 1968 and that the development of software engineering has markedly improved our software. We have a much better understanding of the activities involved in software development. We have developed effective methods of software specification, design and implementation. New notations and tools reduce the effort required to produce large and complex systems.

We know now that there is no single 'ideal' approach to software engineering. The wide diversity of different types of systems and organisations that use these systems means that we need a diversity of approaches to software development. However, fundamental notions of process and system organisation underlie all of these techniques, and these are the essence of software engineering.

Software engineers can be rightly proud of their achievements. Without complex software we would not have explored space, would not have the Internet and modern telecommunications, and all forms of travel would be more dangerous and expensive. Software engineering has contributed a great deal, and I am convinced that, as the discipline matures, its contributions in the 21st century will be even greater.

1.1 FAQs about software engineering

This section is designed to answer some fundamental questions about software engineering and to give you some impression of my views of the discipline. The format that I have used here is the 'FAQ (Frequently Asked Questions) list'. This approach is commonly used in Internet newsgroups to provide newcomers with answers to frequently asked questions. I think that it is a very effective way to give a succinct introduction to the subject of software engineering.

Figure 1.1 summarises the answers to the questions in this section.

1.1.1 What is software?

Many people equate the term *software* with computer programs. However, I prefer a broader definition where software is not just the programs but also all associated documentation and configuration data that is needed to make these programs operate correctly. A software system usually consists of a number of separate programs, configuration files, which are used to set up these programs, system documentation, which describes the structure of the system, and user documentation, which explains how to use the system and web sites for users to download recent product information.

Software engineers are concerned with developing software products, i.e., software which can be sold to a customer. There are two fundamental types of software product:

1. *Generic products* These are stand-alone systems that are produced by a development organisation and sold on the open market to any customer who is able to buy them. Examples of this type of product include software for PCs such as databases, word processors, drawing packages and project management tools.
2. *Customised (or bespoke) products* These are systems which are commissioned by a particular customer. A software contractor develops the software especially for that customer. Examples of this type of software include control systems for electronic devices, systems written to support a particular business process and air traffic control systems.

Question	Answer
What is software?	Computer programs and associated documentation. Software products may be developed for a particular customer or may be developed for a general market.
What is software engineering?	Software engineering is an engineering discipline which is concerned with all aspects of software production.
What is the difference between software engineering and computer science?	Computer science is concerned with theory and fundamentals; software engineering is concerned with the practicalities of developing and delivering useful software.
What is the difference between software engineering and system engineering?	System engineering is concerned with all aspects of computer-based systems development, including hardware, software and process engineering. Software engineering is part of this process.
What is a software process?	A set of activities whose goal is the development or evolution of software.
What is a software process model?	A simplified representation of a software process, presented from a specific perspective.
What are the costs of software engineering?	Roughly 60% of costs are development costs, 40% are testing costs. For custom software, evolution costs often exceed development costs.
What are software engineering methods?	Structured approaches to software development which include system models, notations, rules, design advice and process guidance.
What is CASE (Computer-Aided Software Engineering)?	Software systems which are intended to provide automated support for software process activities. CASE systems are often used for method support.
What are the attributes of good software?	The software should deliver the required functionality and performance to the user and should be maintainable, dependable and usable.
What are the key challenges facing software engineering?	Coping with increasing diversity, demands for reduced delivery times and developing trustworthy software.

Figure 1.1 Frequently asked questions about software engineering

An important difference between these types of software is that, in generic products, the organisation that develops the software controls the software specification. For custom products, the specification is usually developed and controlled by the organisation that is buying the software. The software developers must work to that specification.

However, the line between these types of products is becoming increasingly blurred. More and more software companies are starting with a generic system and customising it to the needs of a particular customer. Enterprise Resource Planning (ERP) systems, such as the SAP system, are the best examples of this approach. Here, a large and complex system is adapted for a company by incorporating information about business rules and processes, reports required, and so on.

1.1.2 What is software engineering?

Software engineering is an engineering discipline that is concerned with all aspects of software production from the early stages of system specification to maintaining the system after it has gone into use. In this definition, there are two key phrases:

1. *Engineering discipline* Engineers make things work. They apply theories, methods and tools where these are appropriate, but they use them selectively and always try to discover solutions to problems even when there are no applicable theories and methods. Engineers also recognise that they must work to organisational and financial constraints, so they look for solutions within these constraints.
2. *All aspects of software production* Software engineering is not just concerned with the technical processes of software development but also with activities such as software project management and with the development of tools, methods and theories to support software production.

In general, software engineers adopt a systematic and organised approach to their work, as this is often the most effective way to produce high-quality software. However, engineering is all about selecting the most appropriate method for a set of circumstances and a more creative, less formal approach to development may be effective in some circumstances. Less formal development is particularly appropriate for the development of web-based systems, which requires a blend of software and graphical design skills.

1.1.3 What's the difference between software engineering and computer science?

Essentially, computer science is concerned with the theories and methods that underlie computers and software systems, whereas software engineering is concerned with the practical problems of producing software. Some knowledge of computer science is essential for software engineers in the same way that some knowledge of physics is essential for electrical engineers.

Ideally, all of software engineering should be underpinned by theories of computer science, but in reality this is not the case. Software engineers must often use *ad hoc* approaches to developing the software. Elegant theories of computer science cannot always be applied to real, complex problems that require a software solution.

1.1.4 What is the difference between software engineering and system engineering?

System engineering is concerned with all aspects of the development and evolution of complex systems where software plays a major role. System engineering is therefore concerned with hardware development, policy and process design and system

deployment as well as software engineering. System engineers are involved in specifying the system, defining its overall architecture and then integrating the different parts to create the finished system. They are less concerned with the engineering of the system components (hardware, software, etc.).

System engineering is an older discipline than software engineering. People have been specifying and assembling complex industrial systems such as aircraft and chemical plants for more than a hundred years. However, as the percentage of software in systems has increased, software engineering techniques such as use-case modelling and configuration management are being used in the systems engineering process. I discuss system engineering in Chapter 2.

1.1.5 What is a software process?

A software process is the set of activities and associated results that produce a software product. There are four fundamental process activities (covered later in the book) that are common to all software processes. These are:

1. *Software specification* where customers and engineers define the software to be produced and the constraints on its operation.
2. *Software development* where the software is designed and programmed.
3. *Software validation* where the software is checked to ensure that it is what the customer requires.
4. *Software evolution* where the software is modified to adapt it to changing customer and market requirements.

Different types of systems need different development processes. For example, real-time software in an aircraft has to be completely specified before development begins whereas, in e-commerce systems, the specification and the program are usually developed together. Consequently, these generic activities may be organised in different ways and described at different levels of detail for different types of software. However, use of an inappropriate software process may reduce the quality or the usefulness of the software product to be developed and/or increase the development costs.

Software processes are discussed in more detail in Chapter 4, and the important topic of software process improvement is covered in Chapter 28.

1.1.6 What is a software process model?

A software process model is a simplified description of a software process that presents one view of that process. Process models may include activities that are part of the software process, software products and the roles of people involved in soft-

ware engineering. Some examples of the types of software process model that may be produced are:

1. *A workflow model* This shows the sequence of activities in the process along with their inputs, outputs and dependencies. The activities in this model represent human actions.
2. *A dataflow or activity model* This represents the process as a set of activities, each of which carries out some data transformation. It shows how the input to the process, such as a specification, is transformed to an output, such as a design. The activities here may represent transformations carried out by people or by computers.
3. *A role/action model* This represents the roles of the people involved in the software process and the activities for which they are responsible.

Most software process models are based on one of three general models or paradigms of software development:

1. *The waterfall approach* This takes the above activities and represents them as separate process phases such as requirements specification, software design, implementation, testing and so on. After each stage is defined it is 'signed-off', and development goes on to the following stage.
2. *Iterative development* This approach interleaves the activities of specification, development and validation. An initial system is rapidly developed from very abstract specifications. This is then refined with customer input to produce a system that satisfies the customer s needs. The system may then be delivered. Alternatively, it may be reimplemented using a more structured approach to produce a more robust and maintainable system.
3. *Component-based software engineering (CBSE)* This technique assumes that parts of the system already exist. The system development process focuses on integrating these parts rather than developing them from scratch. I discuss CBSE in Chapter 19.

I return to these generic process models in Chapter 4 and Chapter 17.

1.1.7 What are the costs of software engineering?

There is no simple answer to this question as the distribution of costs across the different activities in the software process depends on the process used and the type of software that is being developed. For example, real-time software usually requires more extensive validation and testing than web-based systems. However,

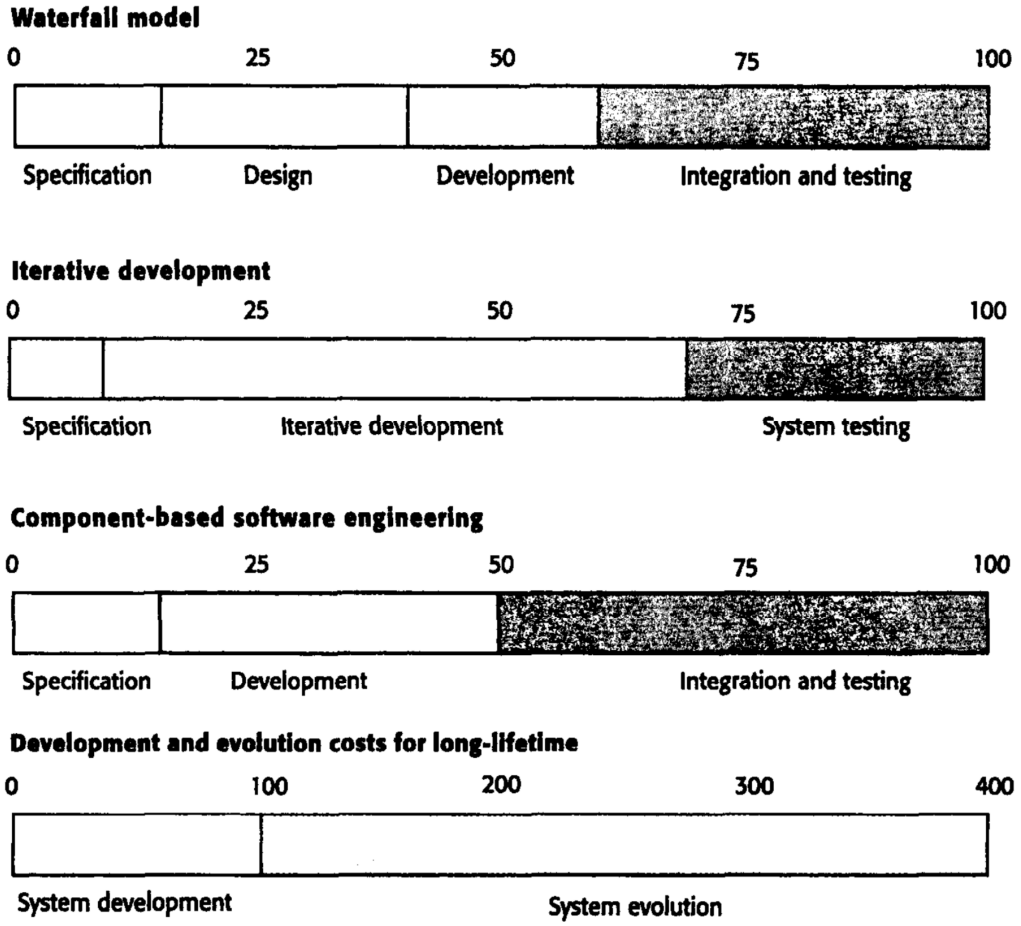


Figure 1.2 Software engineering activity cost distribution

each of the different generic approaches to software development has a different profile of cost distribution across the software process activities. If you assume that the total cost of developing a complex software system is 100 cost units then Figure 1.2 illustrates how these are spent on different process activities.

In the waterfall approach, the costs of specification, design, implementation and integration are measured separately. Notice that system integration and testing is the most expensive development activity. Normally, this is about 40% of the total development costs but for some critical systems it is likely to be at least 50% of the system development costs.

If the software is developed using an iterative approach, there is no hard line between specification, design and development. Specification costs are reduced because only a high-level specification is produced before development in this approach. Specification, design, implementation, integration and testing are carried out in parallel within a development activity. However, you still need an independent system testing activity once the initial implementation is complete.

Component-based software engineering has only been widely used for a short time. We don't have accurate figures for the costs of different software development activities in this approach. However, we know that development costs are reduced

relative to integration and testing costs. Integration and testing costs are increased because you have to ensure that the components that you use actually meet their specification and work as expected with other components.

On top of development costs, costs are also incurred in changing the software after it has gone into use. The costs of evolution vary dramatically depending on the type of system. For long-lifetime software systems, such as command and control systems that may be used for 10 years or more, these costs are likely to exceed the development costs by a factor of 3 or 4, as illustrated in the bottom bar in Figure 1.3. However, smaller business systems have a much shorter lifetime and correspondingly reduced evolution costs.

These cost distributions hold for customised software that is specified by a customer and developed by a contractor. For software products that are (mostly) sold for PCs, the cost profile is likely to be different. These products are usually developed from an outline specification using an evolutionary development approach. Specification costs are relatively low. However, because they are intended for use on a range of different configurations, they must be extensively tested. Figure 1.3 shows the type of cost profile that might be expected for these products.

The evolution costs for generic software products are particularly hard to estimate. In many cases, there is little formal evolution of a product. Once a version of the product has been released, work starts on the next release and, for marketing reasons, this is likely to be presented as a new (but compatible) product rather than as a modified version of a product that the user has already bought. Therefore, the evolution costs are not assessed separately as they are in customised software but are simply the development costs for the next version of the system.

1.1.8 What are software engineering methods?

A software engineering method is a structured approach to software development whose aim is to facilitate the production of high-quality software in a cost-effective way. Methods such as Structured Analysis (DeMarco, 1978) and JSD (Jackson, 1983) were first developed in the 1970s. These methods attempted to identify the basic functional components of a system; function-oriented methods are still used. In the 1980s and 1990s, these function-oriented methods were supplemented by object-oriented (OO) methods such as those proposed by Booch (Booch, 1994) and Rumbaugh (Rumbaugh, et al., 1991). These different approaches have now been integrated into a single unified approach built around the Unified Modeling Language (UML) (Booch, et al., 1999; Rumbaugh, et al., 1999a; Rumbaugh, et al., 1999b).

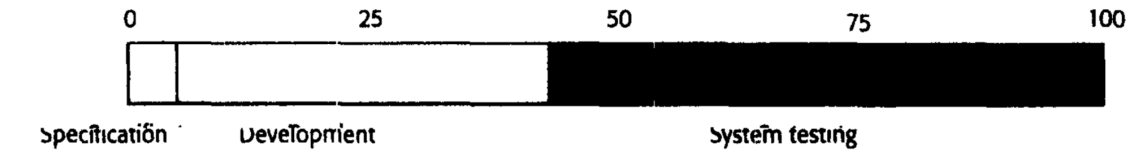


Figure 1.3 Product development costs

Component	Description	Example
System model descriptions	Descriptions of the system models which should be developed and the notation used to define these models.	Object models, data-flow models, state machine models, etc.
Rules	Constraints which always apply to system models.	Every entity in a system model must have a unique name.
Recommendations	Heuristics which characterise good design practice in this method. Following these recommendations should lead to a well-organised system model.	No object should have more than seven sub-objects associated with it.
Process guidance	Descriptions of the activities which may be followed to develop the system models and the organisation of these activities	Object attributes should be documented before defining the operations associated with an object.

Figure 1.4 Method components

There is no ideal method, and different methods have different areas where they are applicable. For example, object-oriented methods are often appropriate for interactive systems but not for systems with stringent real-time requirements.

All methods are based on the idea of developing models of a system that may be represented graphically and using these models as a system specification or design. Methods include a number of different components (Figure 1.4).

1.1.9 What is CASE?

The acronym CASE stands for Computer-Aided Software Engineering. It covers a wide range of different types of programs that are used to support software process activities such as requirements analysis, system modelling, debugging and testing. All methods now come with associated CASE technology such as editors for the notations used in the method, analysis modules which check the system model according to the method rules and report generators to help create system documentation. Th CASE tools may also include a code generator that automatically generates source code from the system model and some process guidance for software engineers.

1.1.10 What are the attributes of good software?

As well as the services that it provides, software products have a number of other associated attributes that reflect the quality of that software. These attributes are not directly concerned with what the software does. Rather, they reflect its behaviour while it is executing and the structure and organisation of the source program and associated documentation. Examples of these attributes (sometimes called non-functional attributes) are the software s response time to a user query and the understandability of the program code.

The specific set of attributes that you might expect from a software system obviously depends on its application. Therefore, a banking system must be secure, an

Figure 1.5 Essential attributes of good software

Product characteristic	Description
Maintainability	Software should be written in such a way that it may evolve to meet the changing needs of customers. This is a critical attribute because software change is an inevitable consequence of a changing business environment.
Dependability	Software dependability has a range of characteristics, including reliability, security and safety. Dependable software should not cause physical or economic damage in the event of system failure.
Efficiency	Software should not make wasteful use of system resources such as memory and processor cycles. Efficiency therefore includes responsiveness, processing time, memory utilisation, etc.
Usability	Software must be usable, without undue effort, by the type of user for whom it is designed. This means that it should have an appropriate user interface and adequate documentation.

interactive game must be responsive, a telephone switching system must be reliable, and so on. These can be generalised into the set of attributes shown in Figure 1.5, which, I believe, are the essential characteristics of a well-designed software system.

1.1.11 What are the key challenges facing software engineering?

Software engineering in the 21st century faces three key challenges:

1. *The heterogeneity challenge* Increasingly, systems are required to operate as distributed systems across networks that include different types of computers and with different kinds of support systems. It is often necessary to integrate new software with older legacy systems written in different programming languages. The heterogeneity challenge is the challenge of developing techniques for building dependable software that is flexible enough to cope with this heterogeneity.
2. *The delivery challenge* Many traditional software engineering techniques are time-consuming. The time they take is required to achieve software quality. However, businesses today must be responsive and change very rapidly. Their supporting software must change equally rapidly. The delivery challenge is the challenge of shortening delivery times for large and complex systems without compromising system quality.
3. *The trust challenge* As software is intertwined with all aspects of our lives, it is essential that we can trust that software. This is especially true for remote software systems accessed through a web page or web service interface. The trust challenge is to develop techniques that demonstrate that software can be trusted by its users.

Of course, these are not independent. For example, it may be necessary to make rapid changes to a legacy system to provide it with a web service interface. To address these challenges, we will need new tools and techniques as well as innovative ways of combining and using existing software engineering methods.

1.2 Professional and ethical responsibility

Like other engineering disciplines, software engineering is carried out within a legal and social framework that limits the freedom of engineers. Software engineers must accept that their job involves wider responsibilities than simply the application of technical skills. They must also behave in an ethical and morally responsible way if they are to be respected as professionals.

It goes without saying that you should always uphold normal standards of honesty and integrity. You should not use your skills and abilities to behave in a dishonest way or in a way that will bring disrepute to the software engineering profession. However, there are areas where standards of acceptable behaviour are not bounded by laws but by the more tenuous notion of professional responsibility. Some of these are:

1. *Confidentiality* You should normally respect the confidentiality of your employers or clients irrespective of whether a formal confidentiality agreement has been signed.
2. *Competence* You should not misrepresent your level of competence. You should not knowingly accept work that is outside your competence.
3. *Intellectual property rights* You should be aware of local laws governing the use of intellectual property such as patents and copyright. You should be careful to ensure that the intellectual property of employers and clients is protected.
4. *Computer misuse* You should not use your technical skills to misuse other people's computers. Computer misuse ranges from relatively trivial (game playing on an employer s machine, say) to extremely serious (dissemination of viruses).

Professional societies and institutions have an important role to play in setting ethical standards. Organisations such as the ACM, the IEEE (Institute of Electrical and Electronic Engineers) and the British Computer Society publish a code of professional conduct or code of ethics. Members of these organisations undertake to follow that code when they sign up for membership. These codes of conduct are generally concerned with fundamental ethical behaviour.

The ACM and the IEEE have cooperated to produce a joint code of ethics and professional practice. This code exists in both a short form, shown in Figure 1.6, and a longer form (Gotterbarn, et al., 1999) that adds detail and substance to the

Software Engineering Code of Ethics and Professional Practice

ACM/IEEE-CS Joint Task Force on Software Engineering Ethics and Professional Practices

PREAMBLE

The short version of the code summarizes aspirations at a high level of the abstraction; the clauses that are included in the full version give examples and details of how these aspirations change the way we act as software engineering professionals. Without the aspirations, the details can become legalistic and tedious; without the details, the aspirations can become high sounding but empty; together, the aspirations and the details form a cohesive code.

Software engineers shall commit themselves to making the analysis, specification, design, development, testing and maintenance of software a beneficial and respected profession. In accordance with their commitment to the health, safety and welfare of the public, software engineers shall adhere to the following Eight Principles:

1. PUBLIC – Software engineers shall act consistently with the public interest.
2. CLIENT AND EMPLOYER – Software engineers shall act in a manner that is in the best interests of their client and employer consistent with the public interest.
3. PRODUCT – Software engineers shall ensure that their products and related modifications meet the highest professional standards possible.
4. JUDGMENT – Software engineers shall maintain integrity and independence in their professional judgment.
5. MANAGEMENT – Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance.
6. PROFESSION – Software engineers shall advance the integrity and reputation of the profession consistent with the public interest.
7. COLLEAGUES – Software engineers shall be fair to and supportive of their colleagues.
8. SELF – Software engineers shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.

Figure 1.6 ACM/IEEE Code of Ethics (©IEEE/ACM 1999)

shorter version. The rationale behind this code is summarised in the first two paragraphs of the longer form:

> *Computers have a central and growing role in commerce, industry, government, medicine, education, entertainment and society at large. Software engineers are those who contribute by direct participation or by teaching, to the analysis, specification, design, development, certification, maintenance and testing of software systems. Because of their roles in developing software systems, software engineers have significant opportunities to do good or cause harm, to enable others to do good or cause harm, or to influence others to do good or cause harm. To ensure, as much as possible, that their efforts will be used for good, software engineers must commit themselves to making software engineering a beneficial and respected profession. In accordance with that commitment, software engineers shall adhere to the following Code of Ethics and Professional Practice.*
>
> *The Code contains eight Principles related to the behaviour of and decisions made by professional software engineers, including practitioners, educators, managers, supervisors and policy makers, as well as trainees and students of*

the profession. The Principles identify the ethically responsible relationships in which individuals, groups, and organizations participate and the primary obligations within these relationships. The Clauses of each Principle are illustrations of some of the obligations included in these relationships. These obligations are founded in the software engineer's humanity, in special care owed to people affected by the work of software engineers, and the unique elements of the practice of software engineering. The Code prescribes these as obligations of anyone claiming to be or aspiring to be a software engineer.

In any situation where different people have different views and objectives, you are likely to be faced with ethical dilemmas. For example, if you disagree, in principle, with the policies of more senior management in the company, how should you react? Clearly, this depends on the particular individuals and the nature of the disagreement. Is it best to argue a case for your position from within the organisation or to resign in principle? If you feel that there are problems with a software project, when do you reveal these to management? If you discuss these while they are just a suspicion, you may be overreacting to a situation; if you leave it too late, it may be impossible to resolve the difficulties.

Such ethical dilemmas face all of us in our professional lives and, fortunately, in most cases they are either relatively minor or can be resolved without too much difficulty. Where they cannot be resolved, the engineer is faced with, perhaps, another problem. The principled action may be to resign from their job, but this may well affect others such as their partner or their children.

A particularly difficult situation for professional engineers arises when their employer acts in an unethical way. Say a company is responsible for developing a safety-critical system and because of time-pressure, falsifies the safety validation records. Is the engineer s responsibility to maintain confidentiality or to alert the customer or publicise, in some way, that the delivered system may be unsafe?

The problem here is that there are no absolutes when it comes to safety. Although the system may not have been validated according to predefined criteria, these criteria may be too strict. The system may actually operate safely throughout its lifetime. It is also the case that, even when properly validated, a system may fail and cause an accident. Early disclosure of problems may result in damage to the employer and other employees; failure to disclose problems may result in damage to others.

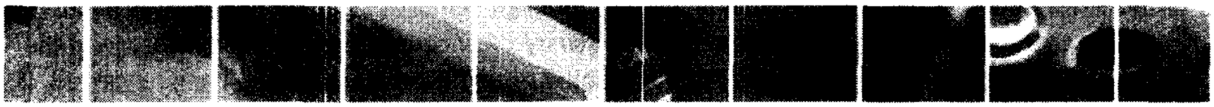
You must make up your own mind in these matters. The appropriate ethical position here depends entirely on the views of the individuals who are involved. In this case, the potential for damage, the extent of the damage and the people affected by the damage should influence the decision. If the situation is very dangerous, it may be justified to publicise it using the national press (say). However, you should always try to resolve the situation while respecting the rights of your employer.

Another ethical issue is participation in the development of military and nuclear systems. Some people feel strongly about these issues and do not wish to participate in any systems development associated with military systems. Others will work on military systems but not on weapons systems. Yet others feel that national security is an overriding principle and have no ethical objections to working on weapons systems.

In this situation it is important that both employers and employees should make their views known to each other in advance. Where an organisation is involved in military or nuclear work, it should be able to specify that employees must be willing to accept any work assignment. Equally, if an employee is taken on and makes clear that he does not wish to work on such systems, employers should not put pressure on him to do so at some later date.

The general area of ethics and professional responsibility is one that has received increasing attention over the past few years. It can be considered from a philosophical standpoint where the basic principles of ethics are considered, and software engineering ethics are discussed with reference to these basic principles. This is the approach taken by Laudon (Laudon, 1995) and to a lesser extent by Huff and Martin (Huff and Martin, 1995).

However, I find their approach is too abstract and difficult to relate to everyday experience. I prefer the more concrete approach embodied in codes of conduct and practice. I think that ethics are best discussed in a software engineering context and not as a subject in their own right. In this book, therefore, I do not include abstract ethical discussions but, where appropriate, include examples in the exercises that can be the starting point for a group discussion on ethical issues.



KEY POINTS

- Software engineering is an engineering discipline that is concerned with all aspects of software production.
- Software products consist of developed programs and associated documentation. Essential product attributes are maintainability, dependability, efficiency and acceptability.
- The software process includes all of the activities involved in software development. The high-level activities of software specification, development, validation and evolution are part of all software processes.
- Methods are organised ways of producing software. They include suggestions for the process to be followed, the notations to be used, system models to be developed and rules governing these models and design guidelines.
- CASE tools are software systems that are designed to support routine activities in the software process such as editing design diagrams, checking diagram consistency and keeping track of program tests that have been run.
- Software engineers have responsibilities to the engineering profession and society. They should not simply be concerned with technical issues.
- Professional societies publish codes of conduct that set out the standards of behaviour expected of their members.